

## AIN 428: INFORMATION RETRIEVAL

The midterm and final assessments, presentation, and project to be used in the 2025–26 academic year are detailed below.

- The midterm submission dates are the same as the presentation dates.
- The final project must be submitted during the final exam week on the date and time announced for the course.
- All submissions must be uploaded to the “midterm” and “final” folders on Piazza.
- The midterm research component requires 3 days, and also 1 day for demo preparation. The final project is estimated to require 10 days of work.
- Both components have equal weight in determining the course grade, each contributing 50%.

### for Midterm

You will deliver a 10-minute presentation on their assigned research topic (7 minutes of explanation + 3 minutes of demo). The required subtopics to be addressed for each research topic are specified. The description and demo suggestions for the code-based demonstration are also provided in the table. The demo code must be viewable in PDF or notebook format, as the classroom computer will be used during presentations. Midterm submissions must be uploaded to the “midterm” folder on Piazza and opened and used there during the presentation.

The table below lists the presentation topics, dates, and assignments for you. Students assigned the same topic are not allowed to present jointly. Additionally, I would like to emphasize that consecutive presentations on the same topic will clearly reflect differences in preparation and depth of understanding.

Grading will be based on 50% of the research topic explanation and 50% of the demo.

| ID | Student Number and Date                  | Topic                                        | Subtopics                                                                                     | Demo                                                                                      | Suggestions for Demo                                                                                                     |
|----|------------------------------------------|----------------------------------------------|-----------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| 1  | 2200765029<br>2210765007<br>(14.04.2026) | Chunk Size Impact on Embedding Retrieval     | Document segmentation theory; Context window constraints; Redundancy vs recall trade-off      | Compare two chunk sizes (small vs large). Analyze Top-3 results for 5 queries.            | Use Sentence-Transformers (all-MiniLM). Small corpus from Wikipedia articles or course PDFs. Keep corpus ≤ 20 documents. |
| 2  | 2210765020<br>2210765021<br>(14.04.2026) | LLM Application Orchestration with LangChain | Pipeline vs agent-based orchestration; Retrieval integration models; Modular LLM architecture | Build a simple RAG pipeline in LangChain. Tool calling optional.                          | Use LangChain + FAISS + OpenAI or local LLM. Keep pipeline simple (Retriever → Prompt → LLM). Avoid multi-agent setup.   |
| 3  | 2210765027<br>2210765029<br>(14.04.2026) | Embedding Model Comparison                   | Embedding dimensionality; Ranking consistency; Latency vs semantic quality                    | Run 5 similarity queries using two embedding models. Compare rankings and inference time. | Use Sentence-Transformers: all-MiniLM vs BGE-small. Measure time with Python time module.                                |
| 4  | 2210765030<br>2210765031<br>(21.04.2026) | Sparse Neural Retrieval (SPLADE)             | Neural lexical expansion; Sparse vs dense indexing; Interpretability differences              | Compare BM25 vs dense embedding retrieval. Conceptually explain sparse neural retrieval.  | Use Rank-BM25 (Python) for BM25. Use Sentence-Transformers for dense baseline. No SPLADE training required.              |
| 5  | 2210765038<br>2210765051<br>(21.04.2026) | Vector Databases & ANN                       | High-dimensional search complexity; HNSW principles; Approximation impact on recall           | Compare brute-force cosine similarity vs FAISS HNSW index latency.                        | Use FAISS with ≤200 vectors. Measure query time difference only.                                                         |

| ID | Student Number and Date                  | Topic                                          | Subtopics                                                                                    | Demo                                                                       | Suggestions for Demo                                                                                            |
|----|------------------------------------------|------------------------------------------------|----------------------------------------------------------------------------------------------|----------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| 6  | 2220765002<br>2220765010<br>(21.04.2026) | Hallucination Detection                        | Retrieval insufficiency vs generative fabrication; Faithfulness; Evidence attribution        | Run 3 RAG queries. Identify unsupported claims manually.                   | Use simple RAG with 10–15 documents. Ask factual questions. Highlight unsupported sentences.                    |
| 7  | 2220765017<br>2220765018<br>(28.04.2026) | Dense Passage Retrieval (DPR)                  | Dual-encoder framework; Query–document embedding alignment; Negative sampling role           | Use pre-trained DPR embeddings for passage retrieval.                      | Use HuggingFace DPR model. Small QA-style dataset (e.g., SQuAD subset). No training required.                   |
| 8  | 2220765021<br>2220765023<br>(28.04.2026) | Negative Sampling in Dense Retrieval           | Contrastive learning intuition; Hard negative concept; Embedding separation                  | Compare cosine similarity of positive vs negative pairs.                   | Create 5 positive and 5 unrelated sentence pairs. Show similarity differences. No loss implementation required. |
| 9  | 2220765043<br>2220765027<br>(28.04.2026) | Multi-Hop Retrieval                            | Compositional query reasoning; Iterative retrieval; Evidence aggregation                     | Decompose 2 complex queries and compare single-step vs two-step retrieval. | Use LLM to rewrite into sub-queries. Use same embedding retriever for both steps.                               |
| 10 | 2220765033<br>2220765034<br>(05.05.2026) | Instruction-Tuned LLMs for Query Understanding | Instruction alignment; Semantic normalization; Retrieval sensitivity to phrasing             | Compare retrieval using raw vs LLM-rewritten query.                        | Use GPT or open-source LLM for rewriting. Keep 3–5 example queries.                                             |
| 11 | 2220765037<br>(05.05.2026)               | Product Quantization in Vector Search          | Subspace vector compression; Approximate similarity; Memory–precision trade-off              | Compare FAISS Flat vs PQ index. Measure latency and Top-3 overlap.         | Use FAISS. ≤300 embeddings. Do not tune PQ parameters deeply — use default settings.                            |
| 12 | 2220765042<br>2220765064<br>(05.05.2026) | XLM-R for Multilingual Retrieval               | Cross-lingual embedding alignment; Zero-shot transfer; Semantic equivalence across languages | Use Turkish queries to retrieve English documents.                         | Use multilingual model (e.g., paraphrase-multilingual-MiniLM). Corpus: English Wikipedia paragraphs.            |
| 13 | 2230765023<br>(12.05.2026)               | BioBERT & ClinicalBERT                         | Domain-adaptive pretraining; Vocabulary specialization; Distributional shift                 | Compare biomedical retrieval using BERT vs BioBERT.                        | Use PubMed abstracts (small subset). Use HuggingFace models. ≤15 documents sufficient.                          |
| 14 | 2230765034<br>(12.05.2026)               | FAISS Index Types                              | Exact vs partition-based indexing; IVF mechanics; Recall-efficiency trade-off                | Compare Flat vs IVF index. Measure latency and Top-3 overlap.              | Use FAISS IVF default settings. Keep dataset small to avoid parameter tuning complexity.                        |
| 15 | 2230765064<br>2240765055<br>(12.05.2026) | Tokenization Impact on Retrieval Quality       | Linguistic segmentation; Term distribution effects; Tokenization impact on ranking stability | Compare whitespace tokenization vs transformer tokenizer on 5 queries.     | Use Python split() for whitespace. Use BERT tokenizer from HuggingFace. Apply BM25 for lexical comparison.      |

## For Final

### Comparative Experimental Study: Multi-Label Toxic Comment Classification Using BERT and a Local LLM

**Problem:** The objective of this project is to conduct a rigorous comparative experimental study on multi-label toxic comment classification. You will investigate two fundamentally different modeling paradigms: a supervised fine-tuned BERT-based classifier and a locally running small Large Language Model used in a prompting-based configuration.

The task is strictly multi-label. Each comment may belong to multiple toxicity categories simultaneously. You are expected to evaluate how a task-specific supervised transformer model compares to a general-purpose on-premise LLM in terms of predictive performance under CPU-only

constraints. The emphasis of this project is not only on implementation but also on methodological rigor, proper experimental design, and the analytical interpretation of results.

**Material:** The dataset to be used is the Jigsaw Toxic Comment Classification dataset. You should use only the official training portion of the dataset and construct your own experimental splits. From this training data, you will generate training, validation, and testing partitions. In addition, the entire study must be conducted using 5-fold cross-validation. Each fold must be treated as an independent experimental cycle, and performance metrics must be reported for each fold separately, as well as averaged across all folds. Given the limited GPU resources, you may reduce the dataset size to improve computational feasibility. However, any subsampling must preserve class distribution and be clearly justified. All preprocessing decisions — including tokenization, length, cleaning, and label handling — must be explicitly documented.

The dataset for this project is the Jigsaw Toxic Comment Classification dataset, originally released as part of a Kaggle competition. You may obtain the dataset from Kaggle (Original Competition Dataset). Please search for “Jigsaw Toxic Comment Classification Challenge” on Kaggle (<https://www.kaggle.com/c/jigsaw-toxic-comment-classification-challenge>). After accepting the competition rules, you can download the file ‘train.csv’.

Only the training file must be used for this project. The official test file should not be used. You are required to construct their own validation and test splits from the training data. Alternatively, the dataset (or closely aligned variants) may be accessed through the Hugging Face Datasets library.

**Method:** The first modeling approach will be a fine-tuned BERT-based classifier. Suitable models include DistilBERT or BERT-base, provided they can be trained within reasonable CPU time constraints. The model must be adapted for multi-label classification using an appropriate output layer. You are expected to describe the pre-processing steps and the training configuration. The implementation must be consistent across cross-validation folds. Model selection decisions must be justified in terms of feasibility and experimental fairness.

The second modeling approach will involve a small LLM running entirely on-premises. You are free to choose any locally executable model, provided it runs on their own machines without external API calls. The LLM must be evaluated under two prompting conditions: zero-shot and few-shot. The few-shot setting must include at least three in-context examples. You must design a consistent, well-structured prompt template and ensure that model outputs are formatted so they can be parsed programmatically into multi-label predictions.

**Experimental Design:** All experiments must follow a 5-fold cross-validation procedure. For each fold, the BERT model will be trained on the training portion and evaluated on the held-out fold. The local LLM will then classify the same held-out fold using both zero-shot and few-shot prompting.

For each fold and toxicity label, you must compute Precision, Recall, and F1-score individually. Per-label performance reporting is mandatory. In addition, you must report macro-averaged and micro-averaged F1-scores across all labels. Performance must be summarized across folds by reporting the mean and standard deviation of each metric. A comparative table presenting BERT, zero-shot LLM, and few-shot LLM results is required.

## **Final Project Report Outline**

## **Multi-Label Toxic Comment Classification: BERT vs Local LLM**

### **1. Dataset Description**

-Source of the dataset (Kaggle Jigsaw Toxic Comment Classification Challenge)-

-Total dataset size used

-Whether subsampling was applied (If subsampling was performed, you must explain how label distribution was preserved)

-Distribution of each label after subsampling (toxic, severe\_toxic, obscene, threat, insult, identity\_hate)

\*A table showing label frequencies is strongly recommended.

### **2. Experimental Design**

-How 5-fold cross-validation was implemented; how training, validation, and test splits were handled inside each fold

-How the usage of the same folds is consistently guaranteed for both BERT and LLM evaluation.

### **3. Fine-Tuned BERT**

-The selected model (e.g., DistilBERT) with the details of architectural parameters, number of epochs, and early stopping strategy (if any).

### **4. Model 2: Local LLM (On-Prem)**

-The chosen LLM model and version, how it was executed locally (e.g., Ollama, llama.cpp)

-Zero-shot prompt design, Few-shot prompt design (minimum 3 examples)

-Output formatting strategy

-How predictions were parsed into six labels

-Prompt templates must be included here or in the appendix.

### **5. Experimental Results**

For each of the 5 folds experiment of BERT, LLM zero-shot, LLM few-shot, you must report:

Per-label Precision, Recall, F1

Macro F1, Micro F1

\*Either a table per fold or a consolidated table is required.

### **6. Conclusion:** What are your observations on BERT and LLM for this task